



Paddle编译与工程效能提升

陈鑫 2021-09
导师:周威

目录

1. 编译基础知识

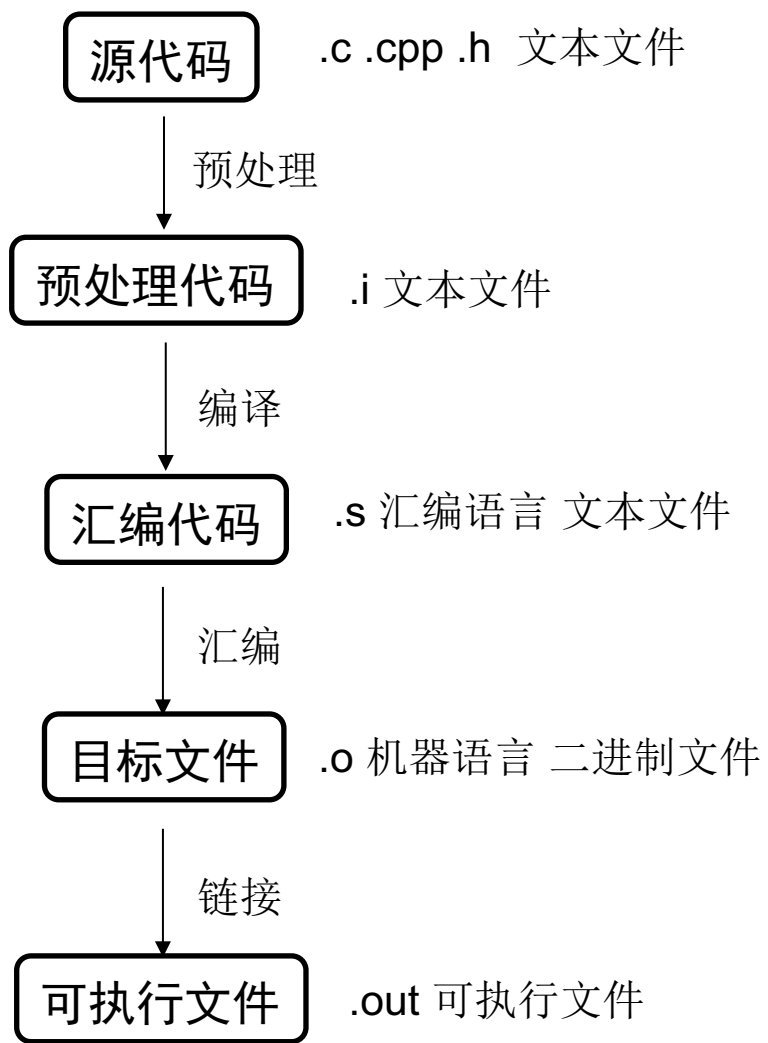
2. Paddle编译过程

3. 编译优化策略

4. Windows-CI相关

5. 疑惑

1. 编译基础知识--编译时发生了什么



预处理：将头文件和宏定义替换成实际内容

编译：将高级语言转换成汇编语言

汇编：将汇编语言转换为机器语言

链接：将目标文件及库文件链接成可执行文件

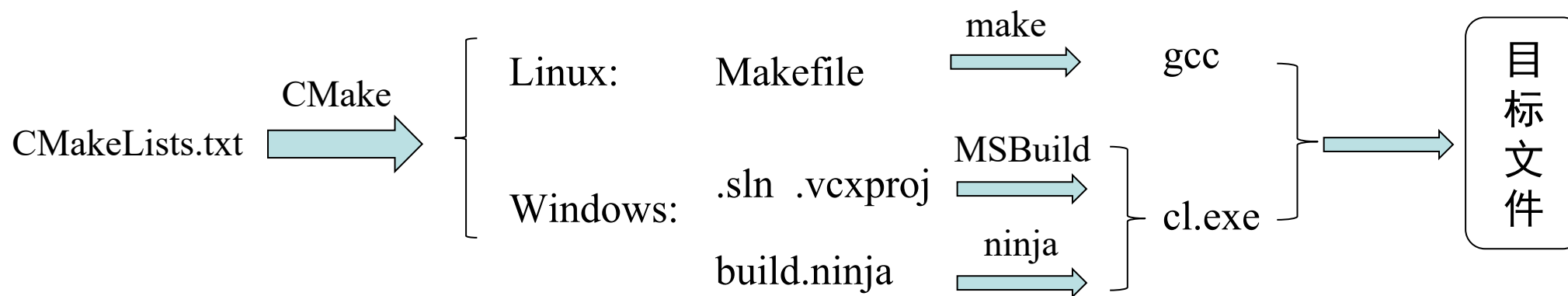
平台	linux	windows
汇编文件	.s	.asm
目标文件	.o .a .so	.obj .lib .dll
可执行文件	.out	.exe
编译工具	gcc (cpp, ccl, as, ld)	vs (cl.exe, link.exe)

1. 编译基础知识—CMake做了什么

指定构建工具，生成对应的原生构建系统，控制编译过程

- 开源
- 跨平台
- 相比makefile等更加高级、方便

```
cmake .. -G Unix Makefile/Ninja/Visual Studio 15 2017 Win64
```



1. 编译基础知识—如何写CMakeLists.txt

- 一个简单的例子

```
cmake_minimum_required(VERSION 3.5)      # cmake最低版本
project(my_project)                      # 项目名称
add_executable(my_exe main.cpp)          # 添加要编译的可执行文件
```

- 包含头文件

```
target_include_directories(
    my_exe                        # 要包含头文件的目标
    ${PROJECT_SOURCE_DIR}/include # 头文件所在目录
)
```

1. 编译基础知识—如何写CMakeLists.txt

- 生成库文件

```
add_library(hello_library          # 库名
            STATIC/SHARED          # 静态/动态
            src/Hello.cpp )        # 源文件

target_include_directories(hello_library # 库文件要包含的头文件
                           ${PROJECT_SOURCE_DIR}/include )

target_link_libraries(my_target # 将库文件链接到目标文件
                      PRIVATE
                      hello_library )
```

PROJECT_SOURCE_DIR: 执行cmake命令的CMakeLists.txt所在路径

PROJECT_BINARY_DIR: 存放构建过程中的临时文件的路径

CMAKE_CURRENT_SOURCE_DIR: 当前处理的 CMakeLists.txt 所在的路径

CMAKE_CURRENT_BINARY_DIR: 当前target 编译目录



目录

1. 编译基础知识

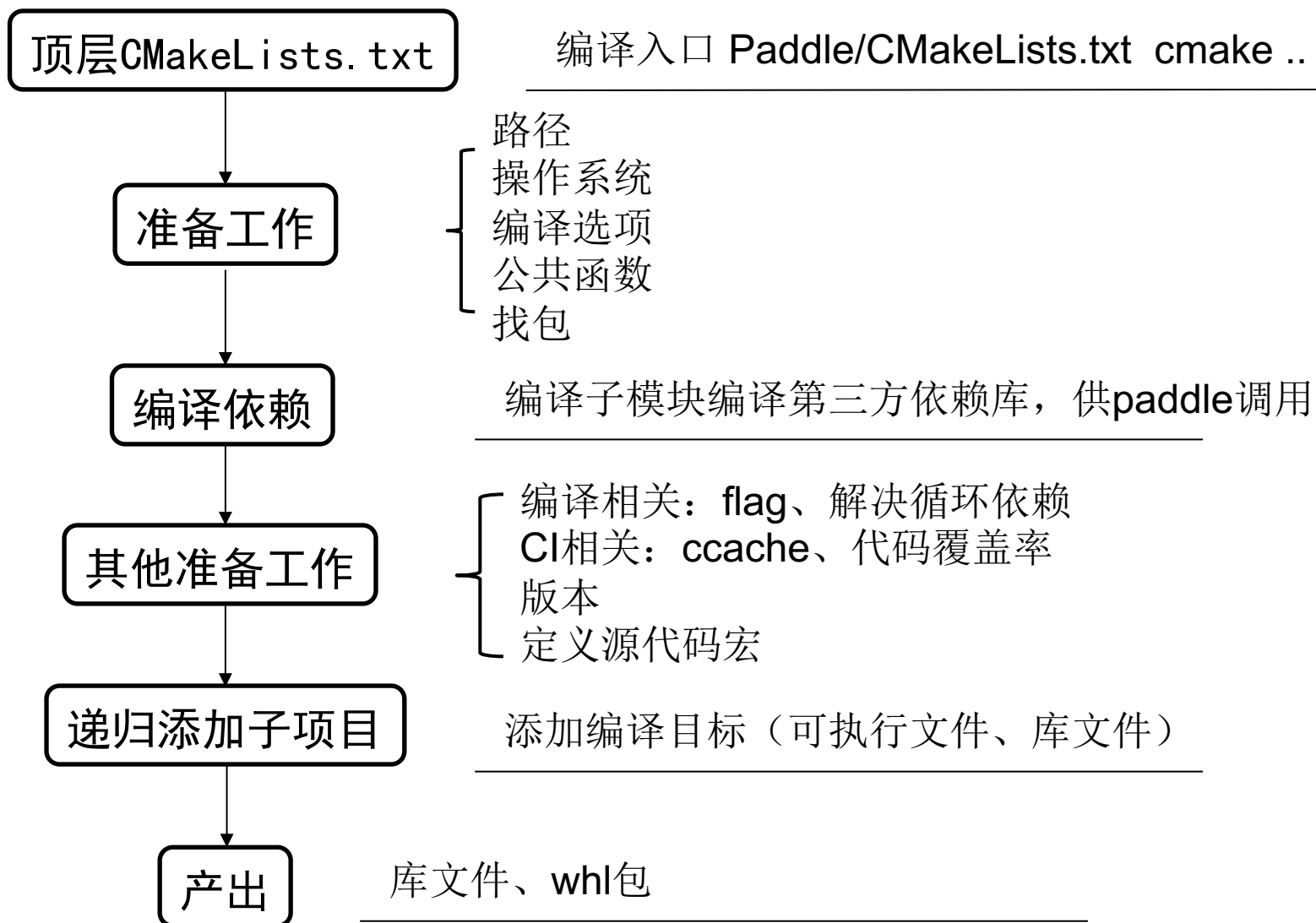
2. Paddle编译过程

3. 编译优化策略

4. Windows-CI相关

5. 疑惑

2. Paddle编译过程—整体流程



2. Paddle编译过程—整体流程

- 设置模块路径

```
set(CMAKE_MODULE_PATH ${CMAKE_MODULE_PATH} "${CMAKE_CURRENT_SOURCE_DIR}/cmake")
```

1. 往CMAKE_MODULE_PATH中添加路径
2. include(module_name)将在CMAKE_MODULE_PATH中搜索<modulename>.cmake文件并执行

- 设置paddle路径

```
set(PADDLE_SOURCE_DIR ${CMAKE_CURRENT_SOURCE_DIR})    # paddle源码路径  
set(PADDLE_BINARY_DIR ${CMAKE_CURRENT_BINARY_DIR})    # 编译产物存放路径
```

2. Paddle编译过程—整体流程

编译选项	含义
WITH_GPU	编译GPU版本的paddle，需要有硬件（NVIDIA显卡）和软件（cuda+cudnn）支持
WITH_TENSORRT	NVIDIA原生推理框架
WITH_ONEMKL	Intel 数学核心库
WITH_SYSTEM_BLAS	系统基础线性代数程序集
WITH_DISTRIBUTE	利用分布式编译
WITH_BRPC_RDMA	使用百度brpc rdma作为rpc框架
ON_INFER	开启推理优化，生成推理库
WITH_NV_JETSON	NVIDIA的嵌入式系统
WITH_PROFILER	谷歌性能测试工具
WITH_INCREMENTAL_COVERAGE	只测试新增代码的覆盖率
WITH_LIBXSMM	用于专门的密集和稀疏矩阵运算的库
WITH_PSLIB	创建PostScript文件的c库

2. Paddle编译过程—整体流程

编译选项	含义
WITH_XBYAK	面向C++ 的x86/x64 JIT 汇编程序，专为高效开发代码而创建的库
WITH_CONTRIB	编译第三方
WITH_PSCORE	参数服务器支持
WITH_INFERENCE_API_TEST	测试推理高级api
WITH_DGC	深度梯度压缩
WITH_NCCL	NVIDIA多gpu通信库
WITH_CCRYPTO	加密算法库
WITH_SW	申威处理器
WITH_MIPS	mips架构，龙芯处理器
WITH_MUSL	轻量级c标准库，用于嵌入式系统和移动设备
WITH_UNITY_BUILD	联合编译，加快编译速度
WITH_STRIP	删除.so文件

2. Paddle编译过程—整体流程

编译选项与宏定义

- `option(option_name “help_text” ON/OFF)`: 提供编译选项，在编译命令行中`cmake .. -Doption_name=ON`，以控制`cmake`分支走向。
- `add_definitions(-Dflag_name)`: 为源文件的编译添加`define` flag，在当前目录和下层目录有效。用来控制源文件的宏的开关

```
# 提供编译选项
option(WITH_GPU “Compile PaddlePaddle with NVIDIA GPU” ${CUDA_FOUND})

# 打开编译开关
cmake .. -DWITH_GPU=ON 或 set WITH_GPU=ON && cmake ..

# 控制编译过程
if(WITH_GPU)
    add_definitions(-DPADDLE_WITH_CUDA)
    FIND_PACKAGE(CUDA REQUIRED)
endif(WITH_GPU)

# 预处理时生效
#ifdef PADDLE_WITH_CUDA
    ...
#endif
```

2. Paddle编译过程—整体流程

- 公共函数

```
include(generic)
```

- cc (CPU) , nv (gpu) , hip (ROCM)

_library, _binary: 添加库文件、可执行文件编译目标，设置依赖和链接库

_test: 编译test的可执行文件，通过add_test()注册测试用例，其中cc_test通过调用cc_test_build和cc_test_run完成这两个步骤

- proto, grpc, brpc

_library: 先调用paddle_protobuf_generate_cpp基于proto文件生成cpp文件，再编译相应的库文件

- python

py_proto: 由proto生成python文件

py_test: 注册python的单测文件

2. Paddle编译过程—整体流程

- `find_package(<PackageName>)` 查找并载入外部项目

```
find_package(CUDA QUIET)           # 非必须，未找到不报错
find_package(Git REQUIRED)          # 必须，未找到报错，进程将终止
```

找包的实质

查找.cmake配置文件并执行，设置指定包的库文件目录、头文件包含目录变量，供编译时使用

找包的过程

1. module模式：寻找Find<package_name>.cmake。两个查找路径：CMAKE_MODULE_PATH--手动编写的配置文件，适用于包安装位置已知的情况； CMAKE_ROOT/ Modules--cmake预定义的配置文件
2. config模式：当module模式失败，或在调用find_package时指定CONFIG|NO_MODULE模式，执行config模式。寻找<package_name>Config.cmake， <lower-case-package-name>-config.cmake，这两种文件都是使用cmake install功能时生成的

结果

```
if(<package_name>_FOUND)    # 表明包是否找到
    <name>_INCLUDE_DIRS: 依赖包的头文件目录
    <name>_LIBRARIES : 依赖包的库文件目录
```

2. Paddle编译过程—第三方库

自己编写Find<package_name>.cmake / 直接找包

find_path(VAR file_name [path1 path2 ...]): 在给定的路径中查找文件，并将包含该文件的路径存放在VAR中

find_library(VAR lib_name [path1 path2 ...]): 在给定路径中查找库文件，并将包含该文件的路径存放在VAR中

```
find_path(OPENBLAS_INC_DIR NAMES cblas.h PATHS ${OPENBLAS_INCLUDE_SEARCH_PATHS})
find_path(OPENBLAS_LAPACK_INC_DIR NAMES lapacke.h PATHS ${OPENBLAS_INCLUDE_SEARCH_PATHS})
find_library(OPENBLAS_LIB NAMES openblas PATHS ${OPENBLAS_LIB_SEARCH_PATHS})
```

2. Paddle编译过程—第三方库

- 组成

必须: zlib, gflags, glog, boost, eigen, threadpool, dlpack, xxhash, warpctc, cblas, protobuf

可选: libxsmm, mklml, mkldnn, python, pybind11, gtest, pslib, box_ps, snappy, leveldb, brpc, libmct, rocksdb, xbyak, dgc, cryptopp (通过编译选项控制)

- 主文件: third_party.cmkae

1. 根据cmake选项, 包含指定的第三方库module
2. 第三方库module调用ExternalProject_Add, 创建外部项目, 完成第三方库的源码下载、解压、编译等过程, 并将编译paddle需要的头文件和库文件复制到third_party/install文件夹
3. 添加自定义目标third_party并依赖包含的第三方库目标, 从而整合为一个整体

```
add_custom_target(third_party ALL DEPENDS ${third_party_deps})
```


2. Paddle编译过程—第三方库

add_custom_target: 添加没有输出的目标，执行某种动作（COMMAND-命令，DEPENDS-依赖其他目标或文件）

add_custom_command: 添加自定义命令，有两种用法

1. OUTPUT模式：通过设置COMMAND, DEPENDS等控制命令执行和文件输出，配合add_custom_target使用

```
add_custom_command(  
    OUTPUT "${CMAKE_CURRENT_BINARY_DIR}/.check_symbol" #是否生成文件取决于命令  
    COMMAND ${CMAKE_COMMAND} -P "${CMAKE_CURRENT_BINARY_DIR}/check_symbol.cmake"  
    DEPENDS paddle_inference_shared)  
#添加依赖于输出文件的目标，并添加到默认构建目标(ALL)  
add_custom_target(check_symbol ALL DEPENDS "${CMAKE_CURRENT_BINARY_DIR}/.check_symbol")
```

2. TARGET模式：在某目标构建前后添加执行命令

PRE_BUILD（编译之前），PRE_LINK（链接之前），POST_BUILD（编译之后）

下面的命令就是在target构建后，删除库文件，再基于libfiles生成静态库

```
add_custom_command(TARGET ${TARGET_NAME} POST_BUILD  
    COMMAND rm "${CMAKE_CURRENT_BINARY_DIR}/lib${TARGET_NAME}.a"  
    COMMAND /usr/bin/libtool -static -o "${CMAKE_CURRENT_BINARY_DIR}/lib${TARGET_NAME}.a"  
    ${libfiles})
```

2. Paddle编译过程—第三方库

ExternalProject_Add：创建自定义目标，以驱动外部项目的下载，更新/补丁，配置，构建，安装和测试

1. 路径：设置外部项目的根目录、源文件目录、安装目录等

2. 下载 download

- URL下载：需要设置URL, URL_HASH / URL_MD5等
- GIT下载：需要设置GIT_REPOSITORY, GIT_TAG等

2. 更新/补丁 update/patch

- 通过UPDATE_COMMAND和PATCH_COMMAND完成

3. 配置 configure

- 外部项目默认为cmake工程，后续构建和安装都默认为cmake
- 通过CONFIGURE_COMMAND <cmd>自定义非cmake命令

4. 构建 build

- 如果配置为非cmake，则默认为make。
- 通过BUILD_COMMAND自定义构建命令
- BUILD_IN_SOURCE 0 / 1：在外部目录 / 下载目录构建
- BUILD_BYPRODUCT:

5. 安装 install

- 如果配置步骤非cmake，则默认为make install
- INSTALL_COMMAND：自定义安装命令

6. 测试 test

- 默认执行外部项目自己的add_test目标
- 只有定义了如下选项之一才会执行测试步骤

TEST_COMMAND / TEST_BEFORE / TEST_AFTER



2. Paddle编译过程—第三方库

- 以gloo为例

```
ExternalProject_Add(  
    extern_gloo  
    ${EXTERNAL_PROJECT_LOG_ARGS}  
    ${SHALLOW_CLONE}  
    GIT_REPOSITORY ${GLOO_REPOSITORY}  
    GIT_TAG ${GLOO_TAG}  
    PREFIX "${GLOO_PREFIX_DIR}"  
    SOURCE_DIR "${GLOO_SOURCE_DIR}"  
    UPDATE_COMMAND ""  
    CONFIGURE_COMMAND ""  
    BUILD_COMMAND mkdir -p ${GLOO_SOURCE_DIR}/build  
    && cd ${GLOO_SOURCE_DIR}/build && cmake .. && make  
    && mkdir -p ${GLOO_LIBRARY_DIR} ${GLOO_INCLUDE_DIR}/gloo  
    INSTALL_COMMAND ${CMAKE_COMMAND} -E copy  
    ${GLOO_SOURCE_DIR}/build/gloo/libgloo.a ${GLOO_LIBRARY_DIR}  
    COMMAND ${CMAKE_COMMAND} -E copy_directory "${GLOO_SOURCE_DIR}/gloo/"  
    "${GLOO_INCLUDE_DIR}/gloo"  
    BUILD_BYPRODUCTS ${GLOO_LIBRARIES})
```

#设置git仓库、tag

自定义构建命令
和安装命令

其他命令

将库文件存放在third_party/install



2. Paddle编译过程—第三方库

最后，将生成的库文件导入，并设置gloo依赖于外部项目目标extern_gloo，从而将主项目与外部项目联系起来，后续目标就可以直接依赖gloo

```
ADD_LIBRARY(gloo STATIC IMPORTED GLOBAL)                                #设置gloo为导入库，全局可见
SET_PROPERTY(TARGET gloo PROPERTY
    IMPORTED_LOCATION ${GLOO_LIBRARIES})                                #设置gloo库从何处导入
ADD_DEPENDENCIES(gloo ${GLOO_PROJECT})                                  #设置gloo库依赖于外部项目
```

2. Paddle编译过程—第三方库

涉及到的cmake函数

头文件相关

- `include_directories(dir)`: 将绝对路径添加到头文件搜索路径，当前CMakeLists的所有target以及在其调用点之后添加的所有子目录中的目标，均会将该路径添加到包含路径中
- `target_include_directories(dir)`: 为特定的target指定头文件包含路径

库文件相关

- `link_directories(dir)`: 添加库文件的搜索路径
- `target_link_libraries(target lib_name)`: 会在库文件的搜索路径中搜索库文件，并设置成target要链接的库
- `link_libraries(lib)`: 链接指定绝对路径的库文件

`add_dependencies(target deps)`: 为target设置依赖，编译器在生成target之前会检查依赖，如果未生成，则会先生成依赖

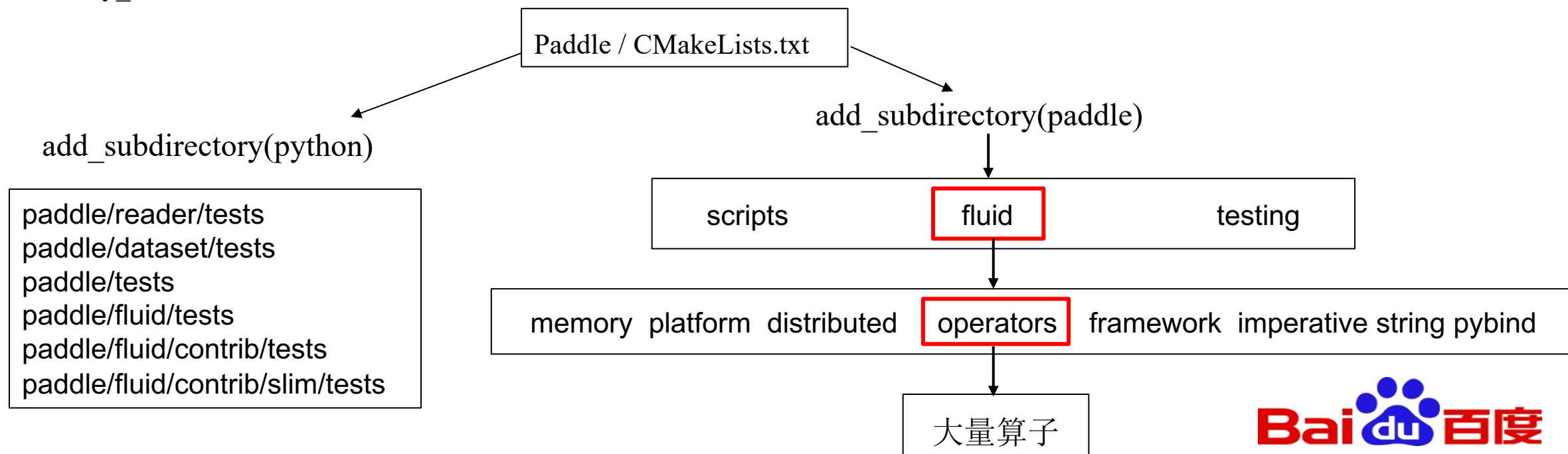
2. Paddle编译过程—operator的旅程

■ 多层CMakeLists.txt

```
add_subdirectory(source_dir [binary_dir] [EXCLUDE_FROM_ALL])
```

为构建添加子目录，执行该路径下的CMakeLists.txt

- source_dir指定CMakeLists.txt和源文件所处路径
- binary_dir指定编译产物的存放位置



2. Paddle编译过程—operator的旅程

子模块与子项目

- `include(module_name)`: 查找模块路径中的`module_name.cmake`文件，加载到原位置执行，相当于头文件
- `add_subdirectory(dir)`: 马上执行`dir`目录下的`CMakeLists.txt`

2. Paddle编译过程—operator的旅程

在paddle/fluid/operators/CMakeLists.txt中调用register_operators函数，传入依赖和要排除的op

#注册了大部分普通OP

```
register_operators(EXCLUDES py_layer_op py_func_op warpctc_op dgc_op ...  
                  DEPS ${OP_HEADER_DEPS})
```

```
function(register_operators) #代码有删减
```

```
    #将当前目录下以op.cc(globbing expressions)结尾的文件的相对路径放到变量OPS中
```

```
    file(GLOB OPS RELATIVE "${CMAKE_CURRENT_SOURCE_DIR}" "*_op.cc")
```

```
    string(REPLACE "_npu" "" OPS "${OPS}")
```

```
    string(REPLACE ".cc" "" OPS "${OPS}")
```

```
    list(REMOVE_DUPLICATES OPS) #只保留一份OP名相同的文件
```

```
    foreach(src ${OPS}) #assign_value_op
```

```
        list(FIND register_operators_EXCLUDES ${src} _index) #判断是否在要排除的OP中
```

```
        if (${_index} EQUAL -1)
```

```
            op_library(${src} #OP名就是TARGET，没有传入SRCS，对于register op，UNITY均为真
```

```
            UNITY DEPS ${register_operators_DEPS})
```

```
        endforeach()
```

```
    if(WITH_UNITY_BUILD)
```

```
        finish_unity_target(cc)
```

```
    endif()
```

```
endfunction()
```


2. Paddle编译过程—operator的旅程

```
function(op_library TARGET) #代码有删减
  set(options UNITY)
  set(multiValueArgs SRCS DEPS)
  if (${op_library_SRCS_len} EQUAL 0)      #普通OP无需传入src
    if (EXISTS ${CMAKE_CURRENT_SOURCE_DIR}/${TARGET}.cc)
      list(APPEND cc_srcs ${TARGET}.cc) #添加当前目录对应OP名的.cc源文件
    endif()
    if(WITH_GPU)
      list(APPEND cu_srcs ${TARGET}.cu) #添加当前目录对应OP名的.cu源文件
    endif()
  endif()
  #将当前目标放入OP_LIBRARY列表中, 该列表缓存在build/CMakeCache.txt, 内部变量, 全局作用域
  set(OP_LIBRARY ${TARGET} ${OP_LIBRARY} CACHE INTERNAL "op libs") ,
  nv_library(${TARGET} #调用对应类型的生成库函数
    SRCS ${cc_srcs} ${cu_srcs} DEPS ${op_library_DEPS} ${op_common_deps})
endfunction()
```

```
//op libs
```

```
OP_LIBRARY:INTERNAL=recurrent_op;eye_op;lstm_op;sync_batch_norm_op;warpctc_op;run_program_op;where_op;where_index_op;var_op
```

```
set(GLOB_OP_LIB ${OP_LIBRARY} CACHE INTERNAL "Global OP library") #生成的OP库将作为其他更高层目标的依赖
cc_library(paddle_pybind SHARED
  SRCS ${PYBIND_SRCS}
  DEPS ${PYBIND_DEPS} ${GLOB_OP_LIB} ${GLOB_OPERATOR_DEPS})
```

2. Paddle编译过程—operator的旅程

位于generic.cmake: 实际添加要生成的目标库, 设置依赖并链接

```
function(nv_library TARGET_NAME) #代码有删减
    set(options STATIC static SHARED shared)
    set(oneValueArgs "")
    set(multiValueArgs SRCS DEPS)
    if (nv_library_SHARED OR nv_library_shared) # 添加动态库目标
        add_library(${TARGET_NAME} SHARED ${nv_library_SRCS})
    endif()
    if (nv_library_DEPS)
        add_dependencies(${TARGET_NAME} ${nv_library_DEPS}) # 为目标库添加依赖和链接
        target_link_libraries(${TARGET_NAME} ${nv_library_DEPS})
    endif()
    foreach(source_file ${nv_library_SRCS})
        string(REGEX REPLACE "\\.[^.]*$" "" source ${source_file}) # 正则表达式怎么用的, 理解不了
        if(EXISTS ${CMAKE_CURRENT_SOURCE_DIR}/${source}.h) # 加上to头文件
            list(APPEND nv_library_HEADERS ${CMAKE_CURRENT_SOURCE_DIR}/${source}.h)
        endif()
    endforeach()
endfunction(nv_library)
```

2. Paddle编译过程—WITH_UNITY联合编译

思想

将同一目录下的源文件分成若干联合组，为每个组生成一个联合文件.cc/.cu，该文件将通过`#include`的方式，包含实际参与编译的op源文件，在预处理时就会把实际的源文件内容放到联合文件中。该组的所有联合文件就是根据该目录起名的联合编译目标的源文件，从而实现联合编译。

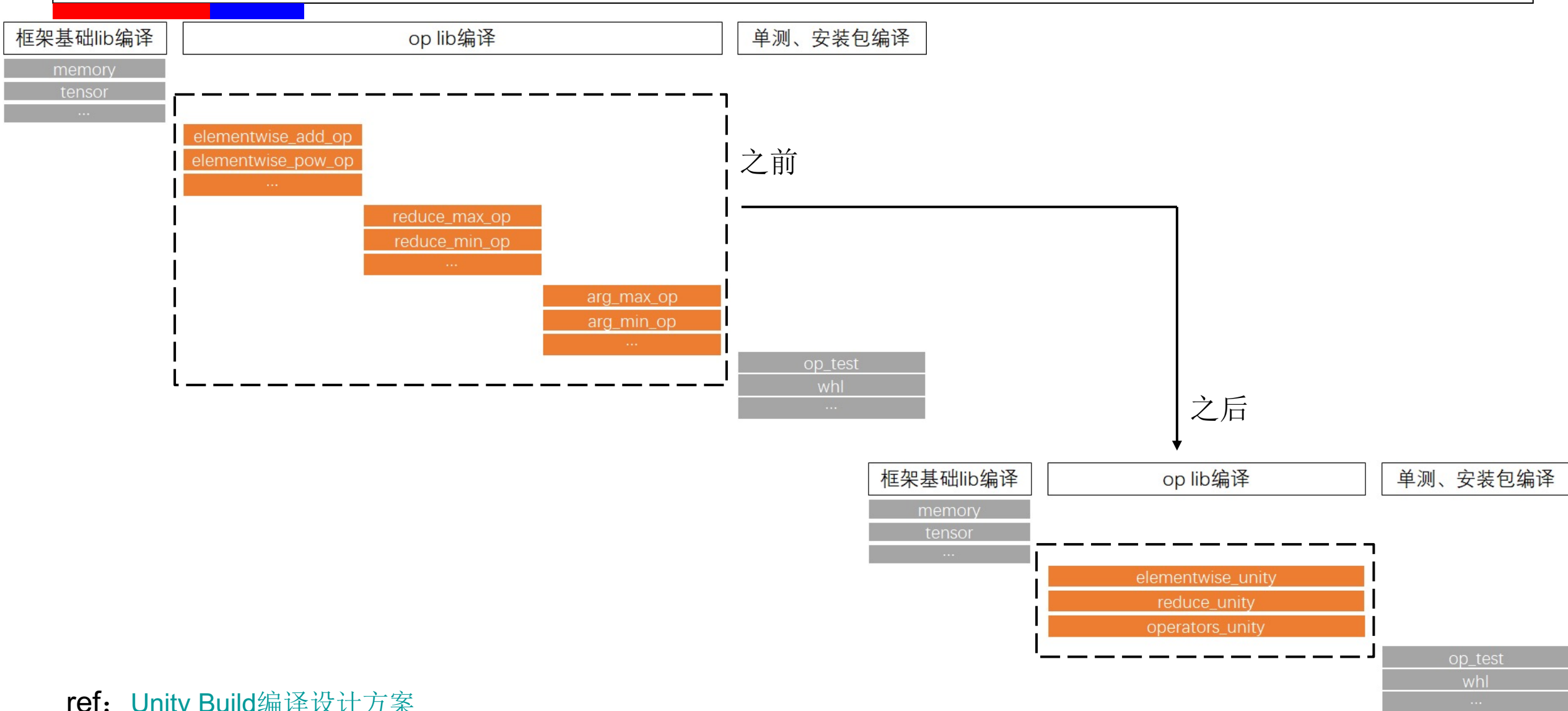
注册联合组 设置联合组源文件、生成联合文件、更新联合组编号

将联合编译目标添加到OP_LIBRARY缓存变量中

设置联合文件的内容与联合编译目标的源文件

将联合文件的内容写入联合文件

2. Paddle编译过程—WITH_UNITY联合编译



ref: [Unity Build编译设计方案](#)

2. Paddle编译过程—WITH_UNITY联合编译

重要变量

UNITY_TARGET-联合编译目标名称

```
string(REPLACE "${PADDLE_SOURCE_DIR}/paddle/fluid/" "" UNITY_TARGET ${CMAKE_CURRENT_SOURCE_DIR})  
string(REPLACE "/" "_" UNITY_TARGET ${UNITY_TARGET})  
set(UNITY_TARGET "paddle_${UNITY_TARGET}_unity")
```

unity_group_index-联合组编号

```
set_property(GLOBAL PROPERTY ${UNITY_TARGET}_${TYPE}_group_index ${unity_group_index})
```

unity_group_sources-联合组的源文件列表

unity_file-联合文件的路径

unity_file_sources-联合文件的内容

unity_target_sources-联合编译目标包含的源文件文件列表

unity_target_\${TYPE}_sources-联合编译目标包含的源文件列表（父作用域）



2. Paddle编译过程—WITH_UNITY联合编译

● register_unity_group-注册联合组

1. 根据当前源文件目录，设置联合编译的目标名称，如在paddle/fluid/operators，则名称为paddle_operators_unity
2. 将联合编译的文件放到unity_group_sources列表中
3. 设置组合后的文件名称并生成空文件，更新组编号

```
function(register_unity_group TYPE)
    string(REPLACE "${PADDLE_SOURCE_DIR}/paddle/fluid/" "" UNITY_TARGET ${CMAKE_CURRENT_SOURCE_DIR})
    string(REPLACE "/" "_" UNITY_TARGET ${UNITY_TARGET})
    set(UNITY_TARGET "paddle_${UNITY_TARGET}_unity") # 设置联合编译的目标名称
    get_property(unity_group_index GLOBAL PROPERTY ${UNITY_TARGET}_${TYPE}_group_index) # 获取当前联合组编号
    set(unity_group_sources ${UNITY_TARGET}_${TYPE}_group_${unity_group_index}_sources)
    set_property(GLOBAL PROPERTY ${unity_group_sources} "")
    foreach(src ${ARGN}) # 将参数中的所有源文件放入unity_group_sources列表中
        set_property(GLOBAL APPEND PROPERTY ${unity_group_sources} ${src})
    endforeach()
    # 设置组合后的文件名称
    set(unity_file ${CMAKE_CURRENT_BINARY_DIR}/${UNITY_TARGET}_${unity_group_index}_${TYPE}.${TYPE})
    if(NOT EXISTS ${unity_file})
        file(TOUCH ${unity_file}) # 先生成空文件
    endif()
    math(EXPR unity_group_index "${unity_group_index} + 1") # 编号加一
    set_property(GLOBAL PROPERTY ${UNITY_TARGET}_${TYPE}_group_index ${unity_group_index}) # 更新全局变量
endfunction(register_unity_group)
```

2. Paddle编译过程—WITH_UNITY联合编译

- function(op_library)

将联合编译目标放入OP_LIBRARY缓存变量中，用联合编译目标作为op的目标名称，后面要编译时需要编译整个联合编译目标

```
if(WITH_UNITY_BUILD AND op_library_UNITY)
  string(REPLACE "${PADDLE_SOURCE_DIR}/paddle/fluid/" "" UNITY_TARGET ${CMAKE_CURRENT_SOURCE_DIR})
  string(REPLACE "/" "_" UNITY_TARGET ${UNITY_TARGET})
  set(UNITY_TARGET "paddle_${UNITY_TARGET}_unity") # 设置联合编译目标名称
  if(NOT ${UNITY_TARGET} IN_LIST OP_LIBRARY) # 如果联合编译目标不在缓存变量中，则加上
    set(OP_LIBRARY ${UNITY_TARGET} ${OP_LIBRARY} CACHE INTERNAL "op libs")
  endif()
endif()

if(WITH_UNITY_BUILD AND op_library_UNITY)
  compose_unity_target_sources(${UNITY_TARGET} cc ${cc_srcs} ${mkldnn_cc_srcs} ${xpu_cc_srcs} ${npu_cc_srcs})
  if(TARGET ${UNITY_TARGET}) # 如果已经存在该联合编译目标，添加该op对应的联合文件作为源文件
    target_sources(${UNITY_TARGET} PRIVATE ${unity_target_cc_sources})
  else() # 否则创建该联合编译目标，设置源文件和依赖
    cc_library(${UNITY_TARGET} SRCS ${unity_target_cc_sources} DEPS ${op_library_DEPS} ${op_common_deps})
  endif()
  add_library(${TARGET} ALIAS ${UNITY_TARGET}) # 用联合编译目标作为op目标的别名
endif()
```

2. Paddle编译过程—WITH_UNITY联合编译

- compose_unity_target_sources-组合联合编译目标的源文件

1. 设置联合后文件的内容：注释、宏定义
2. 某op的源文件被所在联合组的联合文件包含
3. 将联合文件放到unity_target_\${TYPE}_sources中

```
function(compose_unity_target_sources TARGET TYPE)
  foreach(src ${ARGN})
    foreach(unity_group_index RANGE ${unity_group_index_max})
      if(${src_absolute_path} IN_LIST unity_group_sources) # 如果源文件在联合组中
        get_property(set_unity_file_sources GLOBAL PROPERTY ${unity_file_sources} SET)
        if(NOT ${set_unity_file_sources}) # 组合文件的公共开头部分，只需设置一次
          set_property(GLOBAL PROPERTY ${unity_file_sources} "// Generate by Unity Build") # 注释
          set_property(GLOBAL APPEND PROPERTY ${unity_file_sources} ${UNITY_CC_BEFORE_CODE}) # 宏定义
        endif()
        # 包含op的源文件
        set_property(GLOBAL APPEND PROPERTY ${unity_file_sources} "#include \"${src_relative_path}\"")
        set(unity_target_sources ${unity_target_sources} ${unity_file}) # 将组合文件添加到联合编译目标源文件
      endif()
    endforeach()
  endforeach()
  set(unity_target_${TYPE}_sources ${unity_target_sources} PARENT_SCOPE) # 将变量设置为父作用域
endfunction(compose_unity_target_sources)
```


2. Paddle编译过程—WITH_UNITY联合编译

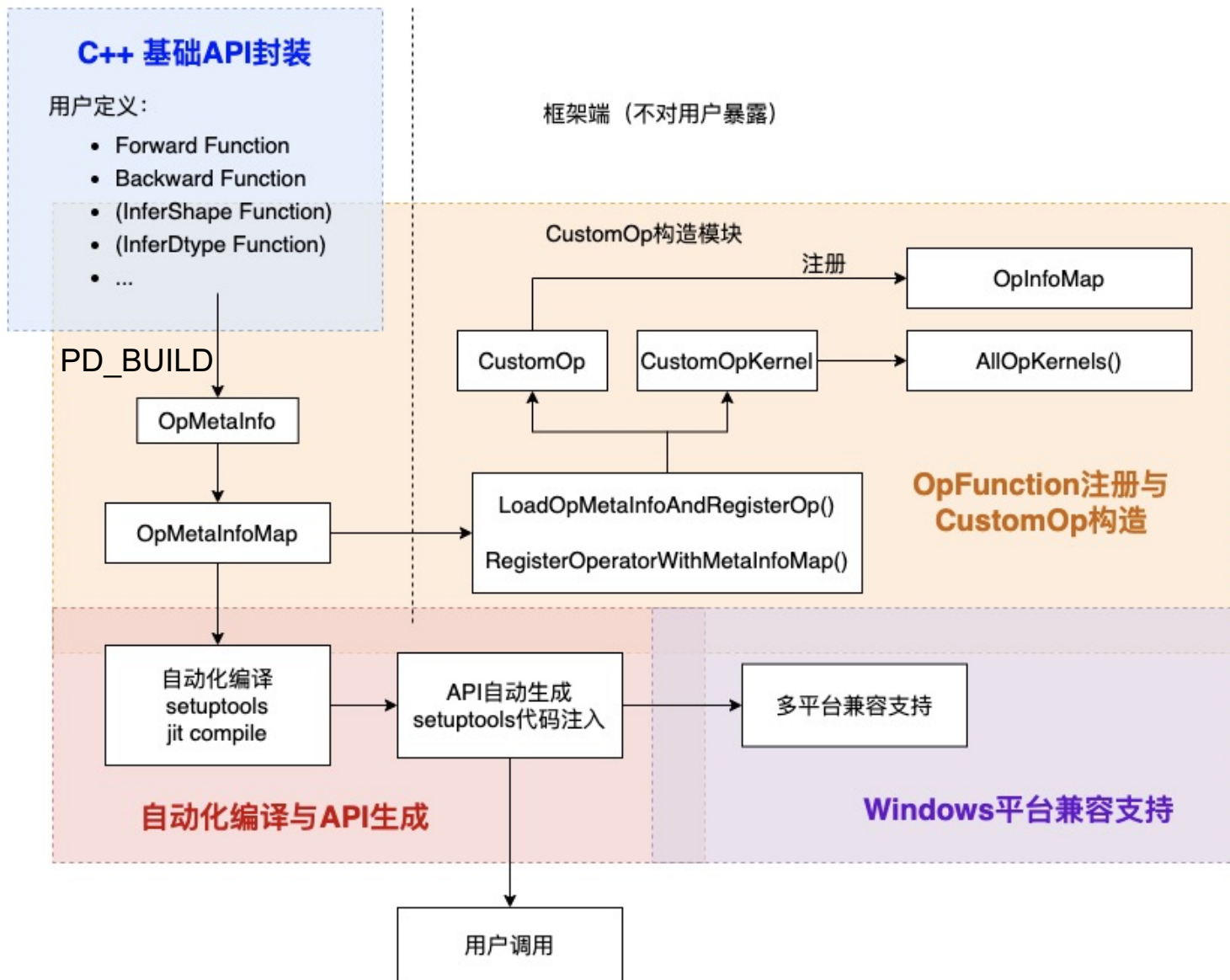
- compose_unity_target_sources

1. 创建联合编译目标，将组合文件添加到联合目标的源文件中
2. 生成组合文件

```
function(finish_unity_target TYPE)
  foreach(unity_group_index RANGE ${unity_group_index_max}) # 生成每个联合组的组合文件
    set(unity_file_read_content "")
    string(JOIN "\n" unity_file_write_content ${unity_file_sources}) # 将联合文件的内容转换为换行的字符串
    file(READ ${unity_file} unity_file_read_content) # 读取联合文件的内容
    if(NOT "${unity_file_read_content}" STREQUAL "${unity_file_write_content}") # 要写的内容和读取到的不相同才写
      file(WRITE ${unity_file} ${unity_file_write_content})
    endif()
  endforeach()
endfunction(finish_unity_target)
```

2. Paddle编译过程—Windows下自定义外部op

用户端



2. Paddle编译过程—Windows下自定义外部op

Windows平台库的导入与导出

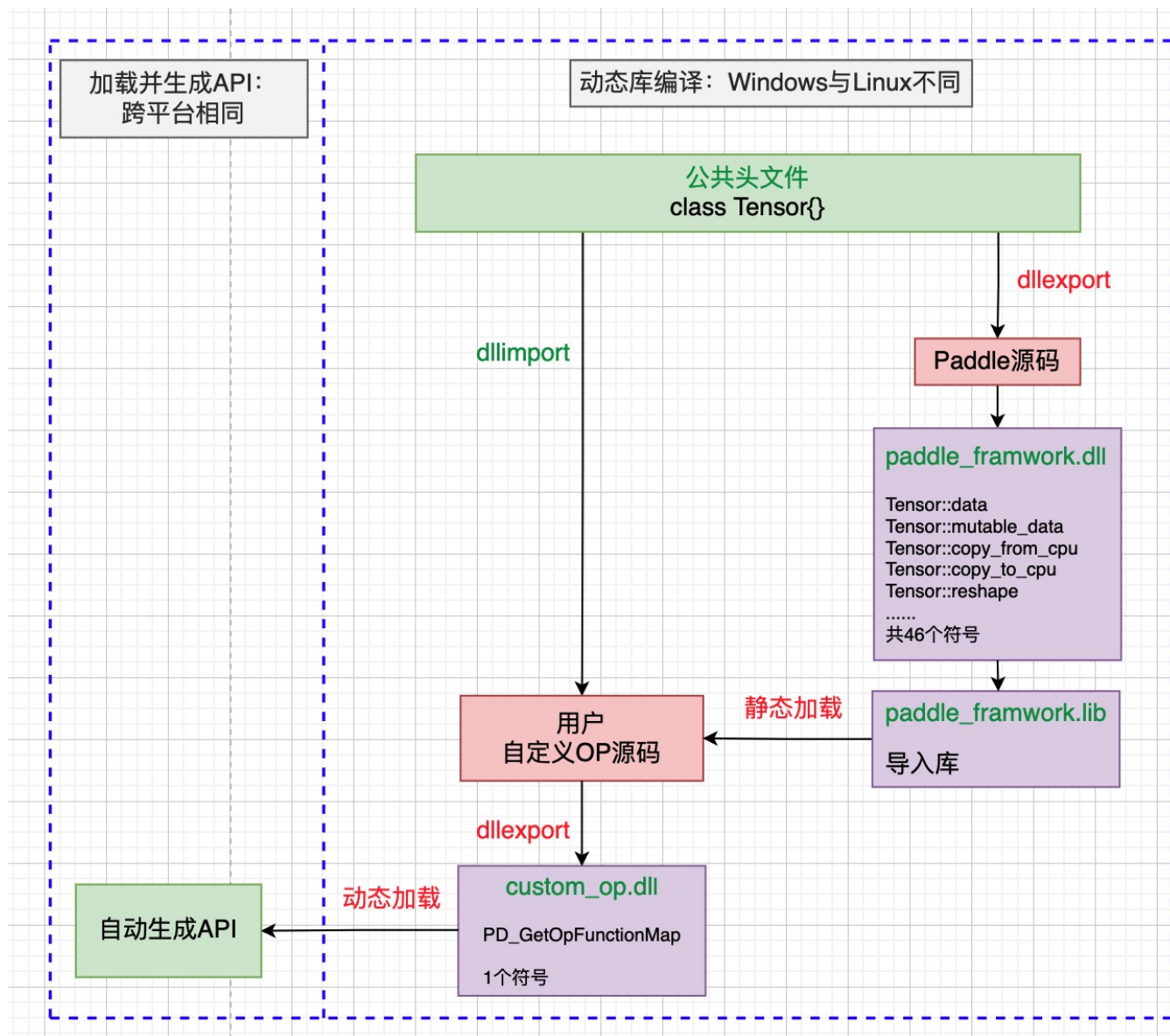
导出: `dllexport`

在类、函数、数据声明时使用, 将其暴露给外部函数使用。

导出: `dllimport`

告诉编译器, 要调用的类、函数、数据位于dll库中

```
#if defined(_WIN32)
#ifndef PADDLE_API
#ifdef PADDLE_DLL_EXPORT
#define PADDLE_API __declspec(dllexport)
#else
#define PADDLE_API __declspec(dllimport)
#endif // PADDLE_DLL_EXPORT
#endif // PADDLE_API
#endif
```



2. Paddle编译过程—Windows下自定义外部op

1. 在自定义OP要用到的类、函数等前面加上dllexport，导出到库

```
class PADDLE_API OpKernelInfo {
```

2. 生成paddle_framework库

```
cc_library(paddle_framework DEPS ${FLUID_FRAMEWORK_MODULES})
```

3. 使用setuptools编译自定义OP源码，在编译命令中指定了core_avx库，最终会生成custom_op_pd.pyd

```
extra_link_args = kwargs.get('extra_link_args', [])
extra_link_args.extend(MSVC_LINK_FLAGS)
lib_core_name = create_sym_link_if_not_exist()
extra_link_args.append('{}'.format(lib_core_name))
if use_cuda:
    extra_link_args.extend(['cudadevrt.lib', 'cudart_static.lib'])
kwargs['extra_link_args'] = extra_link_args
```

2. Paddle编译过程—Windows下自定义外部op

3.1 若为python setup.py install，则会安装custom_op_module。之后可直接通过import custom_op_module使用自定义OP

3.2 若为python setup.py build，则只编译，不安装。之后需静态加载custom_op_pd.pyd，调用PD_GetOpMetaInfoMap函数获得OP信息，注册OP，再生成Python API的方式来使用自定义OP

```
op_names = load_op_meta_info_and_register_op(ext_path)

# generate Python api in ext_path
return _generate_python_module(module_name, op_names, build_directory,
                                verbose)
```

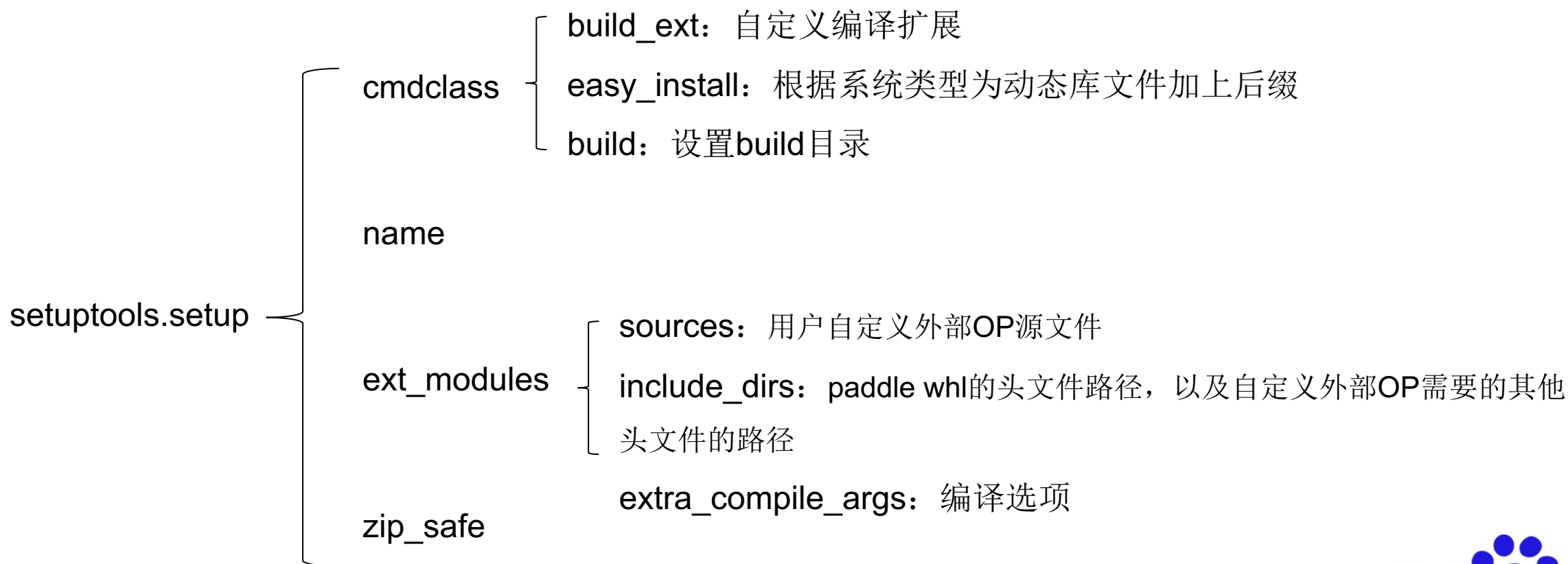
```
// load op api
void LoadOpMetaInfoAndRegisterOp(const std::string& dso_name) {
    void* handle = paddle::platform::dynload::GetOpDsoHandle(dso_name);
    VLOG(3) << "load custom_op lib: " << dso_name;
    typedef OpMetaInfoMap& get_op_meta_info_map_t();
    auto* get_op_meta_info_map =
        detail::DynLoad<get_op_meta_info_map_t>(handle, "PD_GetOpMetaInfoMap");
    auto& op_meta_info_map = get_op_meta_info_map();

    RegisterOperatorWithMetaInfoMap(op_meta_info_map, handle);
}
```

Chen Weihang, a year ago • New custom operator extension mechanism (#3069)

2. Paddle编译过程—Windows下自定义外部op

对setuptools的适配



2. Paddle编译过程—Windows下自定义外部op

build_ext: BuildExtension类对象，继承自setuptools.command.build_ext，复写了build_extensions方法。

以Windows平台为例

- setuptools方法编译安装

1. 检查编译器: check_abi

2. 修改setuptools.command.build_ext生成的编译命令

改为/MT，多线程静态编译；加上一些编译选项；针对.cu文件设置nvcc的编译命令

3. 将.cu文件的目标文件改为.cu.obj，避免与.cc文件的目标文件重复

4. 记录OP信息: op_name, so_name, so_path

5. 调用setuptools.command.build_ext.build_extensions编译

6. 安装

- just-in-time方法编译导入

帮用户写好setup.py文件并运行python setup.py build命令，然后通过加载动态库的方式导入

自定义OP模组，可以在Python里直接调用自定义OP



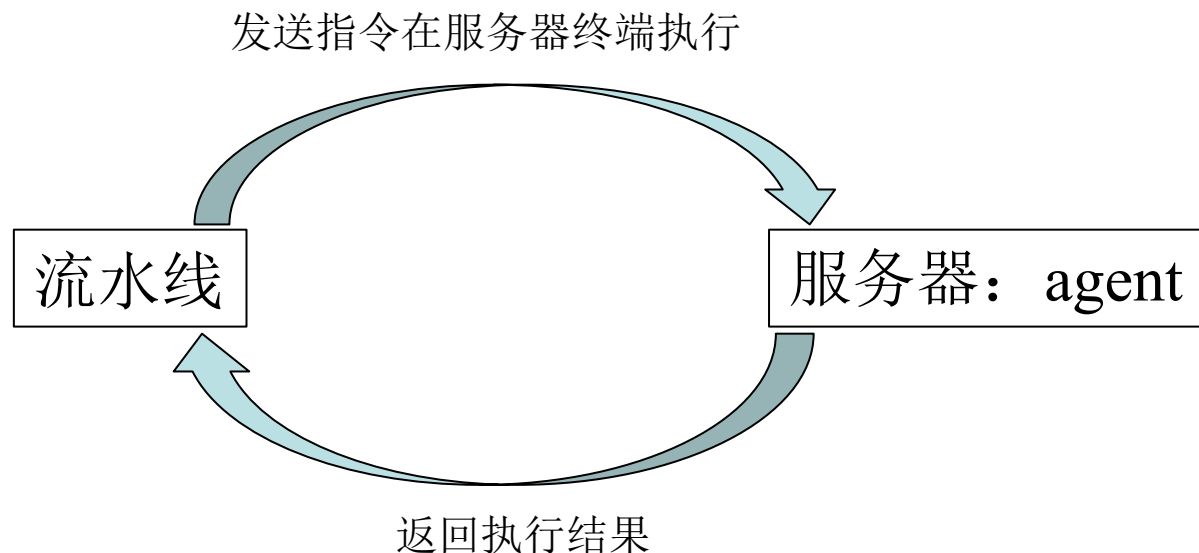
目录

1. 编译基础知识
2. Paddle编译过程

3. CI 基本知识

4. CI 优化策略
5. 疑惑

3. CI基本知识



机器类型: Linux、Windows、kunlun

编译环境: CUDA、cpu、AVX

任务类型: PR-CI、build、release

脚本内容: coverage、inference

主机群: Paddle-windows, Paddle-mac-m1...

单个主机: win11-CUDA 11.1

任务类型: 源信息-分支-消息类型-merge/pr

命令: call paddle_build.bat build_inference_lib

3. CI基本知识-创建与编辑流水线

1. 新建流水线，输入名称与ID

2. 设置源信息：github名称、代码库、分支、消息类型

develop-pr表示将匹配develop分支的每一条pr，配合代码库变更自动触发，就能实现每次提pr都触发CI

编辑源信息 

* 代码源选择

 iCode

 GitHub

 Gitee

* GitHub 连接名称

晓光的github

▼

新建连接

* GitHub代码库

选择已有 ▼

PaddlePaddle/Paddle

▼

分支

消息类型

精确匹配 ▼

develop

pr ▼

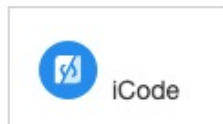


3. CI基本知识-创建与编辑流水线

1. 新建流水线，输入名称与ID
2. 设置源信息：github名称、代码库、分支、消息类型
3. 通用设置：代码库变更自动触发与定时触发、任务超时、跳过流水线、重新运行次数

编辑源信息 𠄎

* 代码源选择



* GitHub 连接名称

晓光的github

新建连接

* GitHub代码库

选择已有 𠄎 PaddlePaddle/Paddle 𠄎

分支

消息类型

精确匹配 𠄎 develop pr 𠄎

- 代码库变更自动触发：每当代码库有变化时，触发流水线。若消息类型为pr，则有新pr时自动触发；若消息类型为merge，则合入新代码时自动触发。
- 任务超时：超过设置时间将失败退出
- 跳过流水线：当commit中包含某字符串时跳过流水线
- 重新运行次数：同一commit能rerun的最大次数

3. CI基本知识-创建与编辑流水线

4. 阶段任务设置

编辑阶段

- 阶段的触发方式
- 阶段的参数：相当于在终端设置的环境变量

编辑插件

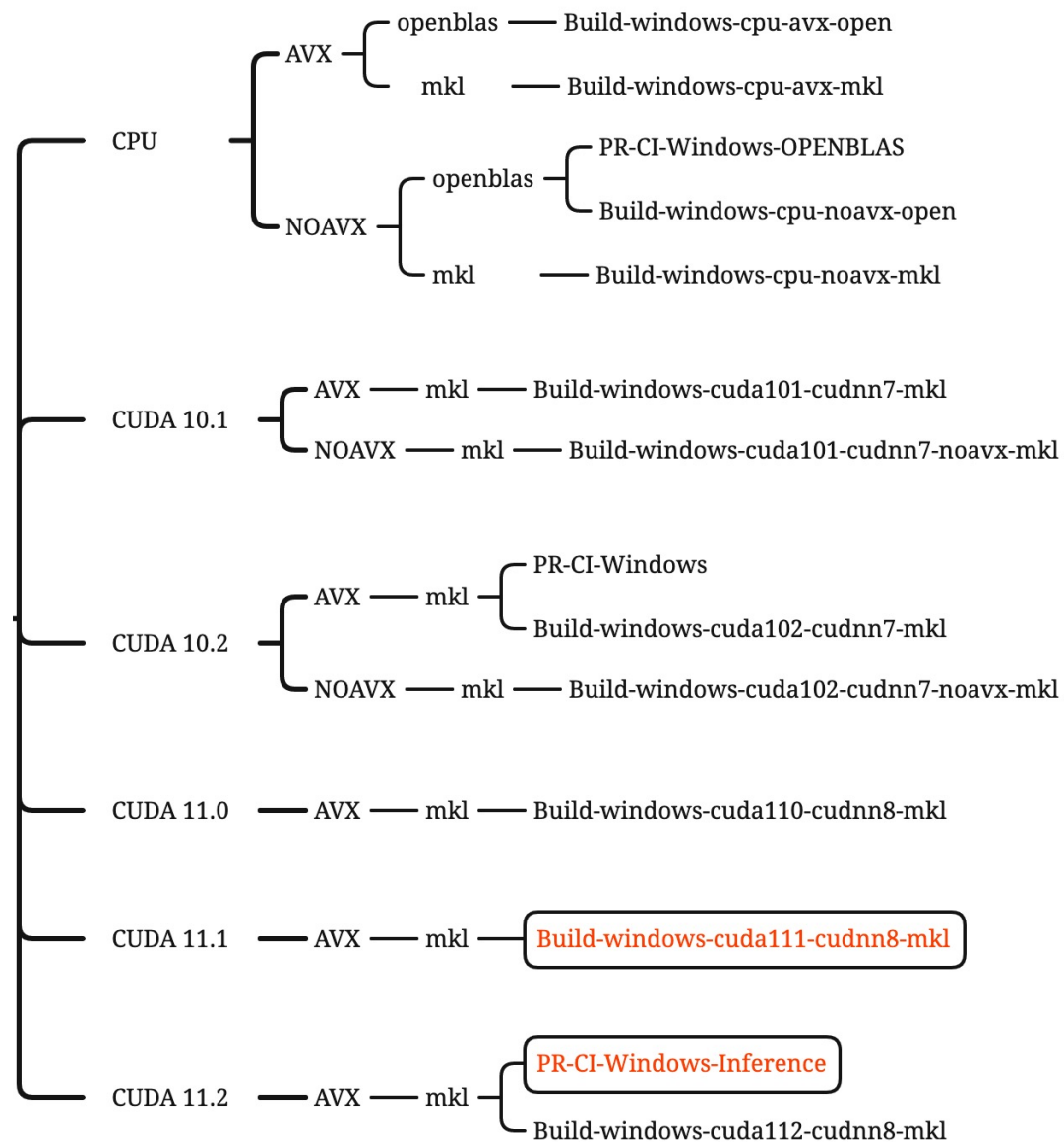
- 脚本类型：sh / bat
- 任务调度权重：在一台机器有多个任务排队时，将优先执行权重高的
- 主机：要运行该流水线的机器，可选机器群，也可选单个机器
- 命令：执行流水线任务的入口，一般做一些清理工作和环境变量设置，然后调用paddle里的脚本
- 产出：打包流水线要用到，设置要上传的产出文件的位置

3. CI基本知识-创建与编辑流水线

技巧

- 若需要在某台机器进行CI测试，且现有流水线主机设置为了该机器所在的机器群，则可以在另一个未被设为主机的机器群新建主机，使机器运行该主机编号的agent，原主机并发数设为0或退出agent，以达到隔离流水线的目的。
- 上述场景的另一种方法为：将原agent离线，新建流水线，设置主机为该单台机器，以该流水线指定pr进行CI测试。

3. CI基本知识-目前CI-Windows流水线汇总



3. CI基本知识-Windows-CI脚本编写

功能	Windows	Linux
设置变量	set BUILD_DIR=build	BUILD_DIR=build
使用变量	echo %BUILD_DIR%	echo \$BUILD_DIR
删除文件	del a.txt	rm a.txt
删除文件夹	rmdir /s /q %BUILD_DIR%	rm -rf %BUILD_DIR%\
函数	:func_name call:func_name vars	function func_name() { func_name vars
条件语句	if cond () else ()	if cond; then do_a else do_b fi
for循环	for /L %i in (1,1,5) do ()	for((i=1;i<=10;i++));do do_a; done
while循环	无while循环, 可用goto + if实现 :while set /a a+=1 if !a! LSS 10 goto:while	while cond; do do_a; done

3. CI基本知识-Windows-CI脚本编写

for循环的几种用法

- 普通循环

```
for /L %i in (1,1,5) do echo %i
```

- 遍历列表

```
for %A in (1 2 3) do @echo %A
```

- 文本处理

```
for /f %i in (test.txt) do echo %i 按行打印
```

delims=, : 指定分隔符

tokens=3: 指定要选取分割后的第几列

```
for /f "delims=, tokens=3" %%i in (a.txt) do echo %%i 以逗号作为分隔符分割txt中的每行，并取第3列
```

shell实现类似的功能

```
cat a.txt | awk -F ',' '{print $3}'
```



3. CI基本知识-Windows-CI脚本编写

call、goto、start的区别

	call	goto	start
相当于	函数调用	跳转	执行
使用范围	命令行，脚本	脚本	命令行，脚本
参数类型	脚本，标签	标签	脚本，可执行程序，文件
运行方式	在同一个进程中执行，变量互通	在同一个进程中执行，变量互通	新开一个窗口（进程）执行，子进程可以读取父进程的变量，反之不行
子程序运行完成后	返回调用点	继续往下执行	开启子进程后，两个进程互不影响

3. CI基本知识-Windows-CI脚本编写

延迟变量扩展: setlocal enabledelayedexpansion

- bat将一行命令, if语句, for语句都看成一条命令, 在执行时解释器会先将命令里所有变量的值先读取, 在执行命令遇到变量时, 直接使用预读取值, 导致变量未按照预期更新。
- 在加上setlocal enabledelayedexpansion, 并通过!!使用变量后, 解释器会在遇到该变量时, 重新读取变量的值, 达到实时更新的目的。

```
if "%WITH_PYTHON%" == "ON" (  
    set a=1  
    echo %a%  
)
```

```
setlocal enabledelayedexpansion  
if "%WITH_PYTHON%" == "ON" (  
    set a=1  
    echo !a!  
)
```

3. CI基本知识-paddle_build.bat

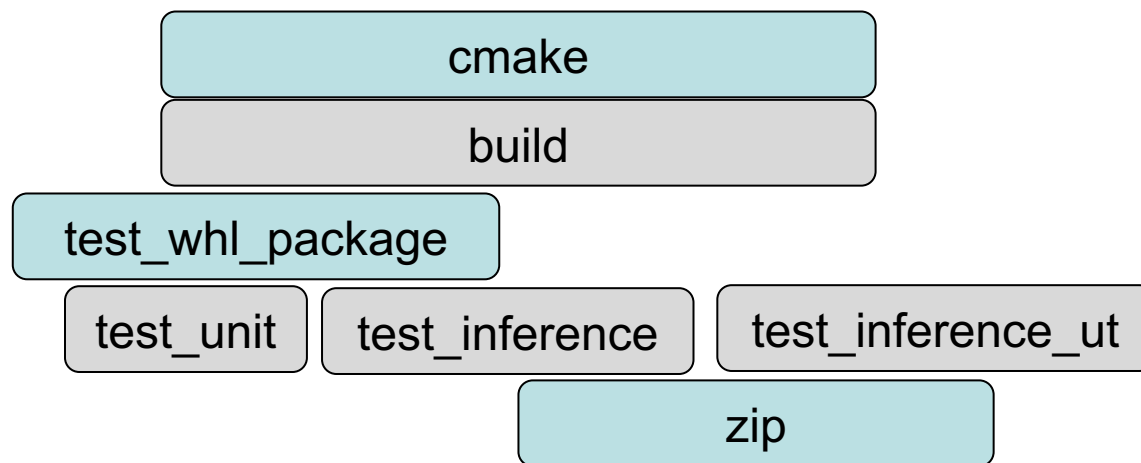
1. 清理进程
2. 安装Python依赖
3. 清理缓存
4. 开启sccache

5. 执行case

CASE_wincheck_mkl
CASE_wincheck_openblas
CASE_wincheck_inference
CASE_build_avx_whl
CASE_build_no_avx_whl
CASE_build_inference_lib

```
222 :CASE_wincheck_mkl
223 set WITH_MKL=ON
224 set WITH_GPU=ON
225 set WITH_AVX=ON
226 set MSVC_STATIC_CRT=OFF
227 set ON_INFER=OFF
228 set WITH_TENSORRT=ON
229
230 call :cmake || goto cmake_error
231 call :build || goto build_error
232 call :test_whl_package || goto test_whl_package_error
233 call :test_unit || goto test_unit_error
234 goto:success
```

case调用函数的流程



目录

1. 编译基础知识

2. Paddle编译过程

3. CI 基本知识

4. CI 优化策略

编译优化

单测优化

5. 疑惑

4. CI优化策略 - 编译优化

● sccache

sccache是与ccache类似的编译缓存工具，使用client-server模式运行，server和client都运行在本地机器上。缓存结果可以存储在本地磁盘或云上,支持C/C++, CUDA等语言的编译缓存。

使用方法：在编译命令前加上sccache。如 `sccache cl main.cpp`，或通过设置 `-DCMAKE_C_COMPILER_LAUNCHER=sccache` 使用存储类型：

本地存储：需要设置 `SCCACHÉ_DIR`（存储路径）和 `SCCACHÉ_CACHE_SIZE`（缓存大小）

云存储：需要设置 `SCCACHÉ_BUCKET`（bucket）、`SCCACHÉ_ENDPOINT`（网址）

`SCCACHÉ_S3_KEY_PREFIX`（bucket下存储目录）、`SCCACHÉ_S3_USE_SSL`（使用传输层安全协议）

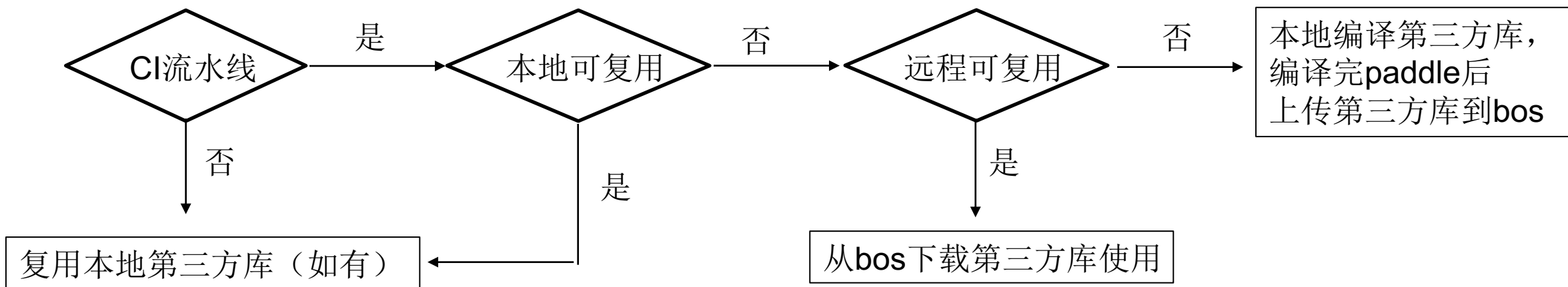
[返回上级](#) | [paddle-windows](#) / [sccache](#) / 0 / 0 / 0

☐	文件名称	存储类型	大小	更新时间	操作
☐	 000000c49b3d35b65eebee6ee8a1791e795756bbd38ad3d7fde7fa7075e1a620	标准存储	190.46KB	2021-12-03 11:09:52	文件信息 复制链接 更多 
☐	 000025de3b2efda53d0205f81a00d31252c2d77269d394a3b98c0d5f99931fb9	标准存储	603.47KB	2021-12-01 16:24:26	文件信息 复制链接 更多 

存储结果

4. CI优化策略 – 编译优化

● 第三方库多机复用



● 实现细节

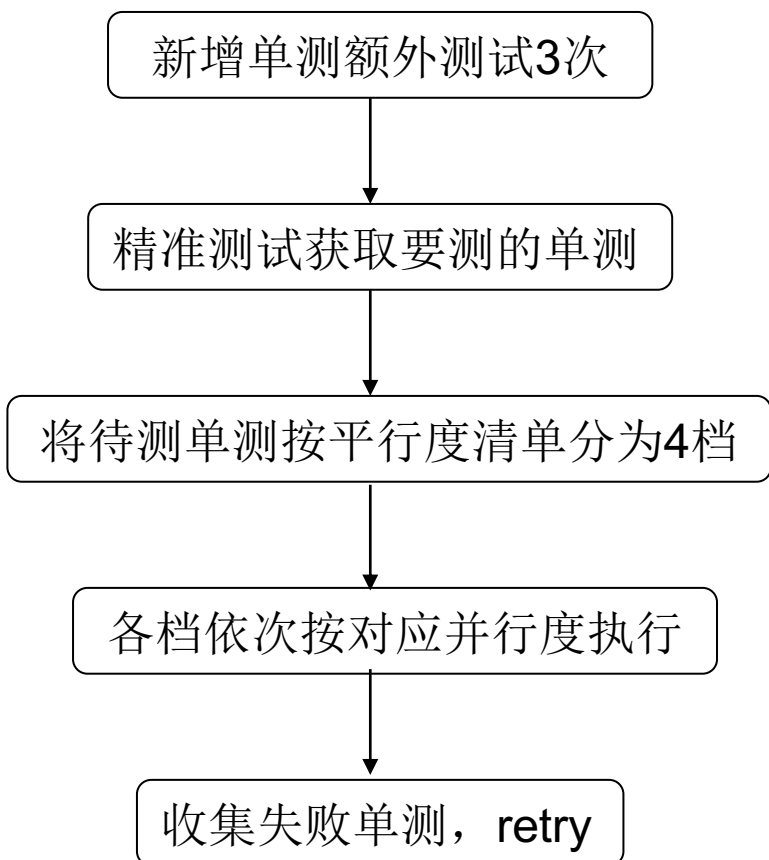
1. 增设第三方库的子目录cpu / cuda101 / cuda102..., 避免不同GPU版本以及GPU与CPU之间的第三方库混用出现问题
2. 根据要执行的case名称, 决定是否多机复用, 避免编包流水线与CI流水线的第三方库混用出现问题

● 收益:

复用远程第三方库的编译时间减少约9min, 粗略计算得触发频率为30%, 所以CI-Windows编译平均耗时减少**3min**

4. CI优化策略 – 单测优化

Paddle/tools/windows/run_unittests.sh



`check_added_ut.sh`: 执行develop分支和当前分支编译时的cmake命令, 再调用ctest获得两个分支的单测列表, 对比得出新增单测

`get_pr_ut.py`, `get_prec_ut_list`: 判断修改文件对所有单测的影响, 只测试受到影响的单测

`parallel_UT_rule.py`: 写有各个平行度的单测清单, 将待测单测按清单分成4档

`run_unittest_gpu $cpu_parallel_job 10`

若存在某单测总失败次数 ≥ 3 , 则单测失败, 退出

4. CI优化策略 – 单测并行策略

Paddle/tools/ parallel_UT_rule.py

1. 在get_prec_ut_list.py中，将跳过的单测打印出来，通过读写文件，将待测单测传递给parallel_UT_rule.py

```
case_to_run = ['test_prec_ut']
for case in all_test_cases_list_new:
    if case in prec_test_cases_list_new:
        case_to_run.append(case)
    else:
        print("{} will not run in PRECISION_TEST mode.".format(case))

with open(file_path, 'w') as f:
    f.write('\n'.join(case_to_run))
```

写入待测单测

```
BUILD_DIR = os.getcwd()
file_path = os.path.join(BUILD_DIR, 'all_ut_list')
with open(file_path, 'r') as f:
    test_cases = f.read()
```

读取待测单测

2. 在parallel_UT_rule.py中，将待测单测按并行度清单，生成4个档位的并行测试单测列表

```
for unittest in high_parallel_job_list:
    if unittest in test_cases:
        high_parallel_job = high_parallel_job + '|' + unittest + '$'
        test_cases.remove(unittest)
```


4. CI优化策略 – 单测并行策略

Paddle/tools/ parallel_UT_rule.py

3. 通过print传回到上一层，再按分号分开

```
if platform.system() == 'Windows':  
    print("{};{};{};{}".format(high_parallel_job, fourth_high_parallel_job,  
                                fifth_high_parallel_job, non_parallel_job))
```

```
output=$(python ${PADDLE_ROOT}/tools/parallel_UT_rule.py)  
cpu_parallel_job=$(echo $output | cut -d ";" -f 1)  
tetrad_parallel_job=$(echo $output | cut -d ";" -f 2)  
two_parallel_job=$(echo $output | cut -d ";" -f 3)  
non_parallel_job=$(echo $output | cut -d ";" -f 4)
```

4. 依次测试各个并行度的单测列表，并排除某些单测

```
ctest -R "$test_case" -E ... -LE ... -j parallel_num
```

结束

- 谢谢！